RFC — Conta compartilhada para EPP: o dinheiro é da sociedade, e cada sócio opera com poder, limite e autoria

TIME Hugo Gomes · Leandro Apolinário Leite · Mateus Carósio
DATA 09/10/2026
VERSÃO 6

Contextualização

Entendendo o problema

O problema de negócio. Uma EPP (empresa de pequeno porte) costuma ter dois, três ou mais sócios. Todos participam da empresa, mas os bancos tratam a empresa como uma conta única acessada por indivíduos. Por isso a gestão do dinheiro da sociedade acontece fora do banco: aprovação por WhatsApp, planilhas, controles paralelos. Ninguém sabe ao certo quem autorizou uma despesa, quanto cada sócio pode movimentar, quem precisa aprovar, nem quanto está destinado a cada finalidade (impostos, folha, fornecedores, freelas). Abrir uma conta PJ qualquer banco faz. O que falta é a **gestão compartilhada do dinheiro da sociedade**.

A ideia do produto. *Não é uma conta para um sócio. É a conta da empresa para todos os sócios.* A unidade do produto é a sociedade empresarial, não o sócio. O dinheiro pertence à empresa; os sócios não são donos de fatias do saldo, eles têm autorização para operar, aprovar ou administrar recursos conforme responsabilidades, permissões, limites e regras de aprovação. O produto não precisa substituir os bancos que a empresa já usa: é uma **camada financeira e de governança da sociedade**, e uma evolução natural é usar Open Finance para consolidar contas e cartões de várias instituições numa visão única. O modelo de negócio é **assinatura recorrente** (acesso à gestão financeira compartilhada) mais **serviços financeiros** (cartões, pagamentos, crédito, investimentos, seguros).

Quem usa e o que o sistema oferece. Os **sócios** (clientes) cadastram a empresa, organizam o dinheiro em responsabilidades, movimentam, programam pagamentos, aprovam pedidos dos outros sócios e consultam o extrato. O **back-office** aprova, bloqueia e encerra a conta. O **parceiro de pagamentos** (gateway simulado) notifica entradas de fora e dispara a execução de pagamentos programados. As operações são: cadastro da empresa, dos sócios e da conta; permissões, limites e regras de aprovação; responsabilidades, reservas, orçamentos e substituições; transação financeira (entrada, saída e transferência interna); pagamento programado com aprovação, cancelamento e execução; e extrato.

Os conceitos do domínio, com um exemplo em que a empresa tem R$ 20.000,00 em conta:

CONCEITO	O QUE É	EXEMPLO
Empresa	Cliente, dona da conta e de todo o dinheiro.	A padaria tem R$ 20.000,00.
Sócio	Pessoa física ligada à empresa. Opera conforme suas permissões. A participação societária não dá direito a parte do saldo.	João, Maria e Ana, sócios.
Responsabilidade	Organização lógica de parte do dinheiro por finalidade, normalmente com um sócio responsável. Não é conta bancária nova.	Freelas, responsável João.
Reserva	Dinheiro da empresa separado logicamente para uma responsabilidade.	R$ 500,00 reservados para Freelas.

CONCEITO	O QUE É	EXEMPLO
Orçamento	Quanto uma responsabilidade pode usar em um período.	Freelas pode usar R$ 800,00 em outubro.
Compromisso	Dinheiro já destinado a um pagamento futuro: continua na conta, mas não está mais disponível.	R$ 400,00 do Pix de sexta.
Pagamento programado	Intenção futura de pagar. Compromete o dinheiro, mas nada saiu.	João programa hoje o Pix de sexta.
Transação	Fato financeiro realizado e imutável. É o único conceito de movimentação: entrada, saída e transferência interna são naturezas; Pix, TED e boleto são meios.	Na sexta o Pix sai e nasce a transação.
Extrato	Visão das transações da conta, não um conceito novo.	Lista de movimentos com autor e responsabilidade.
Permissão	O que um sócio pode fazer (consultar, pagar, aprovar, gerir), em que escopo e até que valor por operação.	Ana só consulta; João paga até R$ 5.000,00.
Regra de aprovação	Acima de que valor um pedido precisa da aprovação de outros sócios, e de quantos.	Até R$ 1.000,00 qualquer sócio paga; acima, duas aprovações.

A garantia. O sistema precisa garantir que **o dinheiro não é criado, perdido nem gasto duas vezes, que nada sai sem alguém autorizado, e que tudo deixa rastro de quem fez o quê**. Se der errado: o saldo diverge do extrato; uma retentativa debita duas vezes; dois pagamentos simultâneos gastam o mesmo saldo ou o mesmo dinheiro já comprometido; um recebimento é creditado duas vezes; um sócio movimenta além do seu limite; um pedido é executado sem as aprovações exigidas; a empresa não consegue reconstruir o saldo a partir das transações. Isso é difícil porque concorrência e retentativa (timeout, duplo clique, notificação repetida do parceiro) são o comportamento normal de um cliente real, e porque agora o dinheiro tem estados (livre, reservado, comprometido) que mudam em vários pedidos ao mesmo tempo.

Fora do escopo desta entrega: login real (a identidade vem do header `X-Pessoa-Id`, então quem souber o id de uma pessoa se passa por ela); integração real com Pix, TED e boleto (o parceiro é um gateway simulado, autenticado só pelo `INTERNAL-TOKEN`); o relógio que dispara a execução na data prevista (a rota de execução existe, quem a chama é um script); liquidação assíncrona da TED, estorno e emissão de boleto; limite diário e permissão temporária para emergência; cartões, Open Finance, crédito, investimentos, seguros e cobrança da mensalidade (roadmap do produto); centro de custo, projeto e categorização automática; encerrar ou renomear responsabilidade e cancelar pedido de reserva; análise real de documentos na aprovação da conta; alteração do quadro societário depois do cadastro; sócio pessoa jurídica ou menor de idade; mais de uma conta por empresa.

Explicando a solução de forma macro

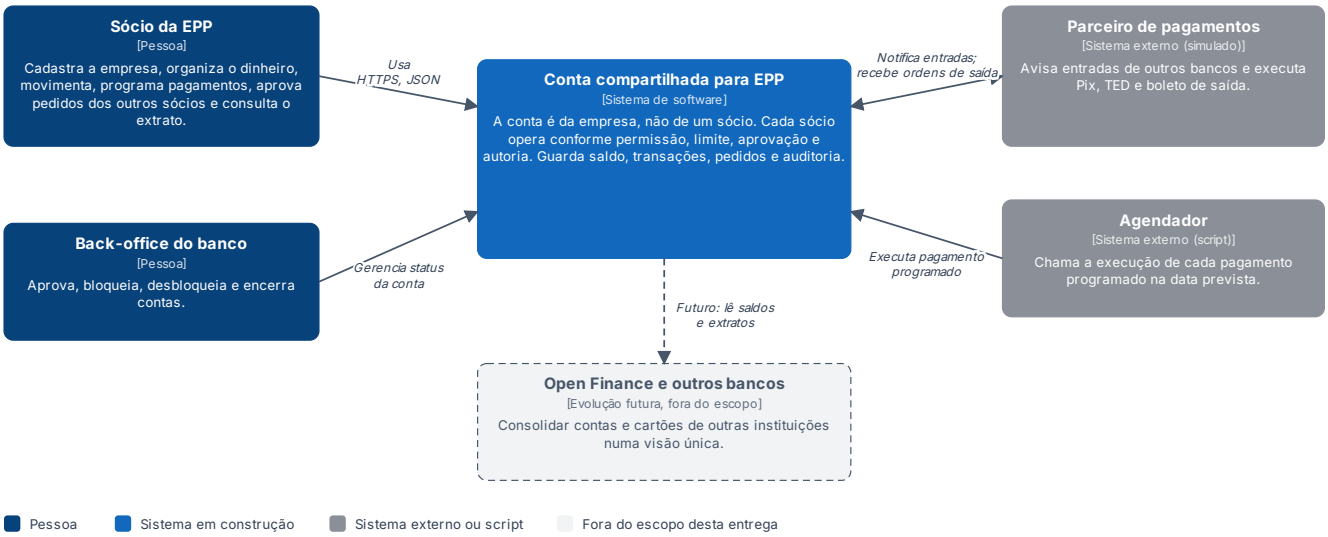
A API segue as camadas do `bootcamp-base-api` (Python, FastAPI, PostgreSQL, Docker): o *resource* valida o formato e escolhe o status HTTP, o *controller* concentra as regras de negócio e dá o único `commit` de cada operação, o *repository* é a única porta para o banco. O desenho junta quatro ideias.

1. **A sociedade é a unidade.** `EMPRESA` tem uma `CONTA`; as pessoas se ligam à empresa pela tabela N:N `EMPRESA_SOCIO`. O que cada sócio pode fazer não vem de um cargo fixo, e sim de `PERMISSAO_SOCIO` (ação, escopo e limite por operação) e de `REGRA_APROVACAO` (acima de X, N aprovações). Todo pedido guarda o vínculo `EMPRESA_SOCIO` do autor, e as mudanças de governança vão para `EVENTO_AUDITORIA`.
2. **O dinheiro é da empresa, organizado em baldes.** A `CONTA` guarda o saldo total e o dividido em **reservado** (separado para responsabilidades, ainda não comprometido) e **comprometido** (destinado a pagamento futuro); o **livre** é o que sobra. Os baldes nunca se sobrepõem, e `RESPONSABILIDADE` e `ORCAMENTO` organizam e limitam o uso sem criar conta bancária nova. `MOVIMENTO_RESERVA` registra cada mexida na reserva.

3. **Intenção não é fato.** `PAGAMENTO_PROGRAMADO` compromete o dinheiro e passa pelas aprovações; só a execução cria a `TRANSACAO`. Entrada de fora, saída para outro banco, transferência interna, e o que o cliente chamaria de depósito, são a mesma `TRANSACAO`, diferenciada por `natureza` e `meio`. O extrato é uma consulta sobre as transações, não uma entidade.
4. **Uma regra de concorrência para tudo.** Toda operação que muda saldo ou pedido começa travando a `CONTA` (`SELECT … FOR UPDATE`, em ordem de id) e só então lê, confere e grava, numa única transação de banco. Idempotência vem de restrições `UNIQUE` somadas a um hash do corpo. Dinheiro é inteiro em centavos (`bigint`); a tarifa vem de `TARIFA`, versionada e imutável; o status da conta guarda o estado atual e um evento por mudança.

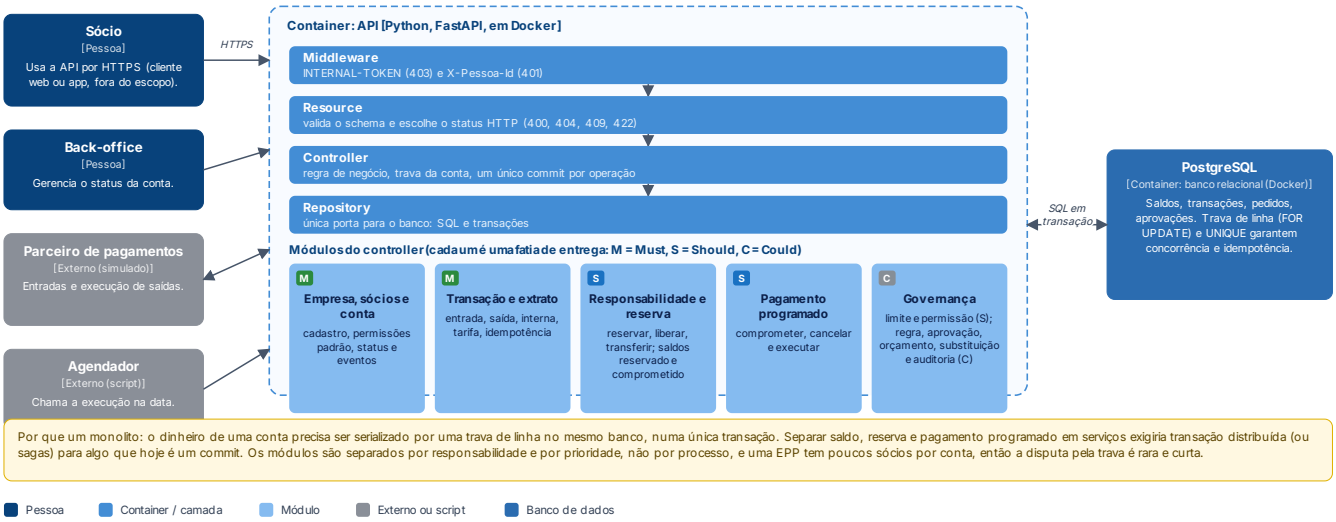
Diagramas de solução (modelo C4). O nível 1 mostra quem usa a plataforma e com quem ela conversa; o nível 2 mostra a API em camadas, os módulos (já marcados com a prioridade de entrega) e o banco. A API é um monolito modular de propósito: o dinheiro de uma conta é serializado por uma trava de linha no mesmo banco, numa única transação, e dividir em serviços exigiria transação distribuída para algo que hoje é um `commit`.

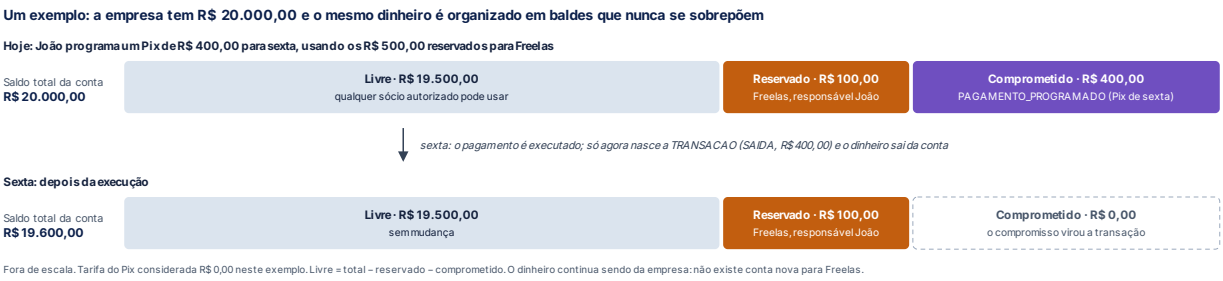
C4 · Nível 1 — Contexto: quem usa a plataforma e com quem ela conversa



Segue o modelo C4 (contexto). Parceiro e agendador são simulados nesta entrega: ambos chamam a API com o INTERNAL-TOKEN.

C4 · Nível 2 — Containers e módulos: um monolito modular sobre um único banco





Como o desenho responde às perguntas centrais do negócio:

PERGUNTA	RESPOSTA DO DESENHO
Quem possui o dinheiro?	A empresa. Sócio não tem fatia do saldo; tem autorização.
Quem pode movimentá-lo?	Os sócios autorizados por `PERMISSAO_SOCIO`, dentro do escopo e do limite.
Como o dinheiro é organizado?	Por responsabilidades, reservas e orçamentos, que são organização lógica dentro da mesma conta.
Como uma operação futura é representada?	Por um `PAGAMENTO_PROGRAMADO`, que já compromete o dinheiro.
Como uma operação realizada é representada?	Por uma `TRANSACAO`, imutável.
Como a empresa sabe quem fez o quê?	Pela autoria em todo pedido, pelas aprovações e pela `EVENTO_AUDITORIA`.
Como a empresa controla operações maiores?	Por limites por sócio e por regras de aprovação.
Como a empresa acompanha a movimentação?	Pelo extrato, derivado das transações.

Priorização da entrega (MoSCoW). São 19 rotas e 16 tabelas, então a entrega é dividida em fatias. Cada fatia termina com a API funcionando e testada, e dá para parar no fim de qualquer uma. Só a fatia *Must* é indispensável para ter uma conta que guarda e movimenta dinheiro com segurança; ela também é o primeiro teste da arquitetura (camadas, trava e idempotência).

PRIORIDADE	FATIA	O QUE ENTREGA	ROTAS	TABELAS
Must	1. Conta e movimentação	Cadastro da empresa com sócios e conta; permissões padrão aplicadas (administrador faz tudo, sócio comum só consulta); status da conta; entrada, saída e transferência interna com tarifa, idempotência e trava; extrato	7: cadastro, consulta de empresa, conta, status, transação (criar e consultar), extrato	9: `PESSOA`, `EMPRESA`, `EMPRESA_SOCIO`, `PERMISSAO_SOCIO`, `CONTA`, `STATUS`, `CONTA_STATUS_EVENTO`, `TARIFA`, `TRANSACAO`
Should	2. Organizar o dinheiro	Responsabilidade, reserva, liberação e transferência de reserva; editar permissão e limite por sócio	4: responsabilidade (criar e consultar), reserva, permissão	2: `RESPONSABILIDADE`, `MOVIMENTO_RESERVA`
Should	3. Pagamento programado	Programar, comprometer, cancelar e executar. Sem regra de aprovação, o pedido nasce `AGENDADO`	4: criar, listar, cancelar, executar	1: `PAGAMENTO_PROGRAMADO`

PRIORIDADE	FATIA	O QUE ENTREGA	ROTAS	TABELAS
Could	4. Governança avançada	Regras e votos de aprovação, orçamento mensal, substituição do responsável, auditoria das mudanças de governança	4: regra, aprovação, orçamento, substituição	4: `REGRA_APROVACAO`, `APROVACAO`, `ORCAMENTO`, `EVENTO_AUDITORIA`
Won't	Fora do escopo	Ver a lista de fora do escopo	—	—

As fatias se encaixam sem retrabalho. As colunas de fatias futuras que já aparecem em tabelas *Must* (`responsabilidade_id` e `pagamento_programado_id` em `TRANSACAO`; saldos reservado e comprometido em `CONTA`) existem desde o início, anuláveis ou zeradas, e as chaves estrangeiras entram com a fatia. Enquanto não há regra de aprovação, `aprovacoes_exigidas` é sempre zero, e `POST /transacao` nunca devolve `APROVACAO_NECESSARIA`. A fatia 3 usa o saldo livre e passa a usar a reserva quando a fatia 2 existe. Se o tempo apertar, o corte é feito de trás para frente: primeiro a fatia 4, depois a 3, depois a 2. O esforço não está estimado em horas, porque depende do tempo do bootcamp e da experiência do time com Python; o tempo real da fatia 1 é a melhor medida para decidir quantas fatias cabem.

O que foi considerado e descartado:

- **`TRANSACAO` com status PENDENTE e EXECUTADA, no lugar de `PAGAMENTO_PROGRAMADO`** — descartada porque a transação passaria a representar algo que ainda não aconteceu: extrato e conferência de saldo teriam de filtrar por status, e uma transação pendente poderia ser cancelada, quebrando a ideia de fato imutável. Ganharia se o produto tratasse a transação como processo longo com muitas etapas (por exemplo, liquidação assíncrona).
- **Conta bancária própria para cada responsabilidade (subconta)** — descartada porque cada subconta teria agência, número e extrato, e reorganizar o dinheiro viraria transferência entre contas, passando a ideia errada de que o dinheiro tem outro dono. Ganharia se a empresa precisasse separar patrimônio de forma legal ou fiscal.
- **Papel fixo (`SOCIO` e `SOCIO_ADMINISTRADOR`) no lugar de permissões e regras** — descartada porque não expressa "até R$ 1.000,00 qualquer sócio paga, acima disso duas aprovações" nem um responsável por área. Ganharia se toda empresa tivesse a mesma estrutura de poder.
- **Tabela de lançamentos separada da transação (partidas dobradas), ou uma tabela por meio ou tipo (depósito, Pix, TED, boleto)** — descartada porque dobra tabelas e escritas, e porque os meios só diferem nos dados da parte externa, que cabem num JSON validado por schema; cada movimento tem no máximo duas pontas, que cabem na mesma linha com o saldo após de cada uma. Ganharia com movimentos de mais de duas contas (rateio) ou estornos parciais, ou se cada meio tivesse regras e consultas próprias.
- **Saldo só em coluna, ou só como soma das transações** — descartada porque a coluna sozinha não deixa rastro (um bug perde dinheiro e o extrato não reconstrói o saldo), e a soma sozinha encarece cada leitura e ainda exige trava. Ganharia num protótipo sem auditoria (coluna) ou de volume baixíssimo (soma).
- **`UPDATE … WHERE saldo >= v` ou trava otimista com versão, no lugar de `FOR UPDATE`** — descartada porque o `UPDATE` condicional não diz o motivo da falha e põe regra de negócio no SQL, e o otimista gera retentativas sob disputa na mesma conta. Ganharia em alto volume com pouca disputa por conta. Numa EPP, com poucos sócios por conta, a espera pela trava é curta, então a trava pessimista é a escolha mais simples e segura.

Implementação

Rotas

Regras comuns. Todas exigem `INTERNAL-TOKEN` (`403` do middleware se ausente ou inválido). As rotas do cliente exigem também `X-Pessoa-Id` (`401` se ausente ou pessoa inexistente) e que a pessoa seja sócia da empresa dona da conta (`403` se não for). Cada ação exige a permissão correspondente em `PERMISSAO_SOCIO` (`403 PERMISSAO_NEGADA`, inclusive quando o escopo `PROPRIAS` não alcança a responsabilidade) e respeita o `limite_transacao_centavos` do sócio (`422 LIMITE_SOCIO_EXCEDIDO`). Cadastro, status da conta, entrada de fora e execução de pagamento programado usam só o token. Critério dos status: `400` formato; `404` recurso inexistente; `409` conflito de unicidade ou idempotência; `422` regra de negócio sobre pedido bem formado, com código no corpo. `GET` é idempotente por natureza. Valores em `valor_centavos`, inteiro ≥ 1. Caminhos no singular, como o base-service (`/sample_entity`), para ter uma convenção só; ações sobre um recurso são sub-recursos (`/aprovacao`, `/cancelamento`, `/execucao`).

Estados. Pagamento programado: `AGUARDANDO_APROVACAO` → `AGENDADO` → `EXECUTADO`; `AGUARDANDO_APROVACAO` → `RECUSADO`; `AGUARDANDO_APROVACAO` ou `AGENDADO` → `CANCELADO`; `AGENDADO` → `FALHOU` (falha definitiva, como boleto vencido). Movimento de reserva: nasce `APLICADO` quando nenhuma regra exige aprovação; senão `AGUARDANDO_APROVACAO` → `APLICADO` ou `RECUSADO`. Os estados finais não mudam mais.

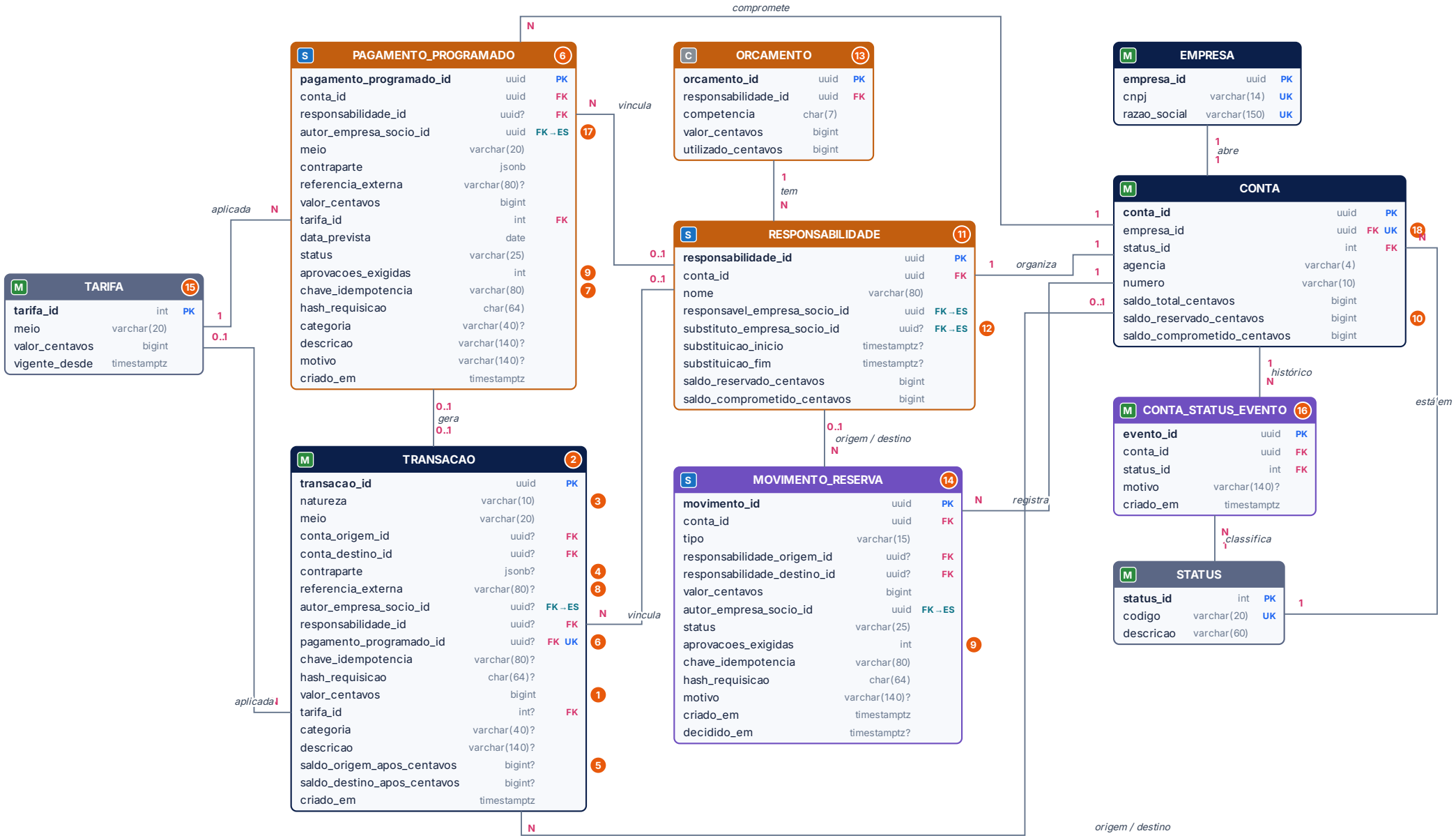
MÉTODO E PRIORIDADE	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`POST` **Must**	`/empresa`	Cadastra a EPP, os sócios, a conta (evento inicial `EM_APROVACAO`) e as permissões padrão, atomicamente. **Não** devolve a mesma resposta ao repetir: `UNIQUE(cnpj)` e `UNIQUE(razao_social)` → `409`	`cnpj`, `razao_social`, `socios[{cpf, nome, email, data_nascimento, papel, participacao_bp}]`	`201` `{empresa_id, conta_id}`; `400` fora do schema; `409` CNPJ, razão social ou e-mail já cadastrado; `422` CNPJ ou CPF com dígito inválido, sócio menor de 18 anos, `ESTRUTURA_SOCIETARIA_INVALIDA`, `SOMA_PARTICIPACAO_INVALIDA`
`GET` **Must**	`/empresa/{empresa_id}`	Empresa, sócios com papel, participação e permissões, e a conta	—	`200`; `401`; `403`; `404`
`GET` **Must**	`/conta/{conta_id}`	Saldo total, reservado, comprometido e livre; status atual e histórico, com `desde`, `ate` e tempo em cada status	—	`200`; `401`; `403`; `404`
`PATCH` **Must**	`/conta/{conta_id}/status`	Back-office aprova, bloqueia, desbloqueia ou encerra: atualiza `status_id` e grava um evento. **Idempotente**: pedir o status que a conta já tem devolve `200` sem novo evento	`status` (`ATIVA`, `BLOQUEADA`, `ENCERRADA`), `motivo`	`200`; `400`; `404`; `422` `TRANSICAO_INVALIDA` (só `EM_APROVACAO` → `ATIVA`, `ATIVA` ↔ `BLOQUEADA`, `ATIVA` ou `BLOQUEADA` → `ENCERRADA`), `SALDO_NAO_ZERADO`
`PUT` `DELETE` **Should**	`/empresa/{empresa_id}/socio/{empresa_socio_id}/permissao/{acao}`	Define ou revoga uma permissão do sócio e grava `EVENTO_AUDITORIA`. **Idempotente**: o `PUT` repetido deixa o mesmo estado (`UNIQUE(empresa_socio_id, acao)`, cria ou atualiza); `DELETE` de permissão que não existe devolve `204`	`escopo` (`PROPRIAS`, `TODAS`), `limite_transacao_centavos` (opcional)	`200`; `204`; `400`; `401`; `403` sem `GERIR_GOVERNANCA`; `404` sócio; `422` `ULTIMO_GESTOR` (tirar `GERIR_GOVERNANCA` do último que a tem), `LIMITE_INVALIDO`

MÉTODO E PRIORIDADE	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`PUT` `DELETE` **Could**	`/empresa/{empresa_id}/regra_aprovacao` e `/regra_aprovacao/{regra_id}`	Cria ou atualiza a regra pela chave natural e grava `EVENTO_AUDITORIA`; só vale para pedidos novos. **Idempotente**: mesma chave `(acao, meio, acima_de_centavos)` atualiza a mesma linha; `DELETE` repetido devolve `204`	`acao`, `meio` (opcional), `acima_de_centavos`, `exige_todos` ou `aprovacoes_necessarias`	`200` `{regra_id}`; `204`; `400`; `401`; `403`; `404`; `422` `REGRA_INATINGIVEL` (mais aprovações do que sócios aprovadores possíveis), `PARAMETROS_INCONSISTENTES`
`POST` **Should**	`/responsabilidade`	Cria a responsabilidade com seu responsável. Repetir devolve `409` por `UNIQUE(conta_id, nome)`	`conta_id`, `nome`, `responsavel_empresa_socio_id`	`201` `{responsabilidade_id}`; `400`; `401`; `403`; `404` conta; `409` nome repetido na conta; `422` `RESPONSAVEL_NAO_E_SOCIO`
`GET` **Should**	`/responsabilidade/{responsabilidade_id}`	Responsável, substituto vigente, saldo reservado e comprometido, orçamento do mês	—	`200`; `401`; `403`; `404`
`PUT` **Could**	`/responsabilidade/{responsabilidade_id}/substituicao`	Define o substituto e a janela. **Idempotente**: o `PUT` repetido deixa o mesmo estado	`substituto_empresa_socio_id`, `inicio`, `fim`	`200`; `400`; `401`; `403`; `404`; `422` `PERIODO_INVALIDO`, `SUBSTITUTO_INVALIDO`
`PUT` **Could**	`/responsabilidade/{responsabilidade_id}/orcamento/{competencia}`	Define o teto do mês (`AAAA-MM`). **Idempotente**: `UNIQUE(responsabilidade_id, competencia)`, cria ou atualiza	`valor_centavos`	`200`; `400` competência malformada; `401`; `403`; `404`; `422` `ORCAMENTO_ABAIXO_DO_UTILIZADO`
`POST` **Should**	`/movimento_reserva`	Reserva (livre → responsabilidade), libera (responsabilidade → livre) ou transfere entre responsabilidades. Aplica na hora ou fica aguardando aprovação, conforme a regra. **Idempotente**: `UNIQUE(autor_empresa_socio_id, chave_idempotencia)` e `hash_requisicao`	header `Idempotency-Key`; `tipo`; `responsabilidade_origem_id` e `responsabilidade_destino_id` (conforme o tipo); `valor_centavos`; `motivo`	`201` (`APLICADO` ou `AGUARDANDO_APROVACAO`); `200` repetição idêntica; `400`; `401`; `403`; `404`; `409` chave reutilizada com corpo diferente; `422` `TIPO_INCOERENTE`, `SALDO_LIVRE_INSUFICIENTE`, `RESERVA_INSUFICIENTE`, `LIMITE_SOCIO_EXCEDIDO`, `CONTA_NAO_ATIVA`
`POST` **Must**	`/transacao`	Movimenta dinheiro na hora, por Pix, TED, boleto ou internamente: saída para outro banco, transferência interna ou entrada de fora (simulada pelo gateway). A `natureza` não vem no corpo: sai da presença de origem e destino. **Idempotente**: saída e interna por `UNIQUE(autor_empresa_socio_id, chave_idempotencia)`; entrada e boleto por `UNIQUE(meio, referencia_externa)`	header `Idempotency-Key` (saída e interna); `meio`; `valor_centavos`; `conta_origem_id` e/ou `conta_destino_id`; `contraparte` (JSON da parte externa); `referencia_externa` (boleto e entrada); `responsabilidade_id` (opcional); `categoria`, `descricao`	`201`; `200` repetição idêntica (mesma resposta, saldo intacto); `400`; `401`; `403` `PERMISSAO_NEGADA`; `404` conta; `409` chave ou referência reutilizada com corpo diferente, boleto já pago; `422` `CONTA_NAO_ATIVA`, `LIMITE_SOCIO_EXCEDIDO`, `APROVACAO_NECESSARIA` (a regra exige aprovação: use `/pagamento_programado`), `ORCAMENTO_EXCEDIDO`, `RESERVA_INSUFICIENTE` (com responsabilidade), `SALDO_LIVRE_INSUFICIENTE` (sem responsabilidade), `TRANSACAO_SEM_CONTA_NOSSA`, origem igual ao destino, boleto vencido
`GET` **Must**	`/transacao/{transacao_id}`	Uma transação	—	`200`; `401`; `403` pessoa sem vínculo com a origem nem com o destino; `404`

MÉTODO E PRIORIDADE	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`GET` **Must**	`/conta/{conta_id}/extrato`	Visão das transações em que a conta é origem ou destino, da mais recente à mais antiga, paginada por cursor `(criado_em, transacao_id)`. Cada item traz data, natureza, meio, contraparte, valor com sinal, tarifa, categoria, descrição, autor, responsabilidade, `pagamento_programado_id` e saldo após	`limite` (1–100), `cursor`, `de`, `ate`, `categoria`, `natureza`, `meio`, `responsabilidade_id`, `autor_empresa_socio_id`	`200` `{itens, proximo_cursor}`; `400` limite fora da faixa, cursor malformado ou data inválida; `401`; `403`; `404`
`POST` **Should**	`/pagamento_programado`	Programa uma saída (Pix, TED ou boleto) para uma data: **compromete** valor + tarifa na hora, congela a tarifa e fixa `aprovacoes_exigidas`. Nasce `AGENDADO` (sem regra) ou `AGUARDANDO_APROVACAO`. **Idempotente**: `UNIQUE(autor_empresa_socio_id, chave_idempotencia)` e `hash_requisicao`; boleto também por `UNIQUE(meio, referencia_externa)` parcial nos pedidos ativos	header `Idempotency-Key`; `meio`; `valor_centavos`; `contraparte`; `referencia_externa` (boleto); `data_prevista`; `responsabilidade_id` (opcional); `categoria`, `descricao`	`201`; `200` repetição idêntica; `400`; `401`; `403`; `404`; `409` chave reutilizada com corpo diferente, boleto já em pedido ativo; `422` `DATA_NO_PASSADO`, `CONTA_NAO_ATIVA`, `LIMITE_SOCIO_EXCEDIDO`, `ORCAMENTO_EXCEDIDO`, `RESERVA_INSUFICIENTE`, `SALDO_LIVRE_INSUFICIENTE`, `REGRA_INATINGIVEL` (menos aprovadores elegíveis do que o exigido), boleto vencido
`GET` **Should**	`/conta/{conta_id}/pagamento_programado`	Lista os pagamentos programados da conta, com status e aprovações recebidas, paginada por cursor	`status`, `limite`, `cursor`	`200` `{itens, proximo_cursor}`; `400`; `401`; `403`; `404`
`POST` **Could**	`/pagamento_programado/{id}/aprovacao` e `/movimento_reserva/{id}/aprovacao`	Registra o voto de um sócio elegível (com `APROVAR` no escopo, diferente do autor). Ao atingir `aprovacoes_exigidas` o pedido avança (`AGENDADO` ou `APLICADO`); uma recusa encerra o pedido e devolve o dinheiro. **Idempotente**: `UNIQUE(empresa_socio_id, alvo)`, mesmo voto repetido devolve `200`	`decisao` (`APROVADA`, `RECUSADA`)	`201`; `200` voto repetido idêntico; `400`; `401`; `403` `PERMISSAO_NEGADA`; `404`; `409` `VOTO_JA_REGISTRADO` com decisão diferente; `422` `AUTOR_NAO_APROVA_O_PROPRIO`, `PEDIDO_NAO_PENDENTE`
`POST` **Should**	`/pagamento_programado/{id}/cancelamento`	O autor, ou quem tem `GERIR_RESPONSABILIDADE`, cancela um pedido pendente ou agendado e devolve o dinheiro comprometido. **Idempotente**: cancelar o que já está `CANCELADO` devolve `200`	`motivo`	`200`; `401`; `403`; `404`; `422` `JA_EXECUTADO`, `ESTADO_FINAL` (recusado ou falhou)

MÉTODO E PRIORIDADE	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`POST` **Should**	`/pagamento_programado/{id}/execucao`	Parceiro ou agendador executa o pedido `AGENDADO` na data: cria a `TRANSACAO` e baixa o comprometido. **Idempotente**: `UNIQUE(pagamento_programado_id)` em `TRANSACAO`, repetir devolve `200` com a mesma `transacao_id`	—	`201`; `200` já executado; `404`; `422` `NAO_AGENDADO` (pendente, recusado ou cancelado), `DATA_NAO_CHEGOU`, `CONTA_NAO_ATIVA` (o pedido continua `AGENDADO`), boleto vencido (o pedido vira `FALHOU` e devolve o dinheiro)

Banco de Dados (Somente diagrama)



Por quê? Legenda dos números do diagrama

1. **Por que _centavos nos nomes?**
   Dinheiro é guardado como inteiro de centavos (8000 = R$ 80,00). Decimal e float erram no arredondamento (0,1 + 0,2 não dá 0,3). Vale para valor, saldo, tarifa, limite e orçamento.

2. **Por que só TRANSACAO (sem LANCAMENTO, DEPOSITO, PIX...)?**
   Depósito, recebimento, pagamento e transferência são o mesmo fato: dinheiro mudou de lugar. Uma tabela só, sem conceitos equivalentes; o que varia são os atributos natureza e meio.

3. **natureza e meio**
   natureza é o sentido do dinheiro (ENTRADA, SAIDA, INTERNA); meio é o canal (PIX, TED, BOLETO, INTERNO). Um novo meio é um novo valor, não uma nova tabela nem uma nova rota.

4. **Como entra ou sai dinheiro de outro banco?**
   A ponta que não é conta nossa fica nula e é descrita em contraparte (jsonb, validado por schema do meio). Sem origem = entrada; sem destino = saída; as duas = transferência interna.

5. **saldo_*_apos**
   Foto do saldo total da conta logo após o movimento. O extrato mostra saldo corrido sem somar tudo, e o saldo é reconstruível e auditável (entradas − saídas − tarifas).

6. **PAGAMENTO_PROGRAMADO ≠ TRANSACAO**
   Programado é intenção futura: já compromete o dinheiro, mas nada saiu. Transação é fato realizado e imutável. A UK em pagamento_programado_id garante no máximo uma transação por agendamento.

7. **chave_idempotencia + hash_requisicao**
   Repetir a mesma chave com o mesmo corpo devolve o mesmo resultado, sem efeito novo. A mesma chave com corpo diferente é conflito (409). Protege contra timeout e duplo clique.

8. **UK (meio, referencia_externa)**
   Identifica o fato externo (boleto, notificação do parceiro): o mesmo boleto não é pago duas vezes e a mesma notificação não credita duas vezes. No agendamento a UK é parcial: só pedidos ativos.

9. **aprovacoes_exigidas**
   Foto da regra de aprovação no momento do pedido. Mudar a regra depois não altera pedidos em andamento. Zero = não precisa de aprovação.

10. **Saldos da CONTA**
    total = dinheiro da empresa. reservado = separado para responsabilidades e ainda não comprometido. comprometido = destinado a pagamento futuro. livre = total − reservado − comprometido (derivado). São baldes que nunca se sobrepõem.

11. **RESPONSABILIDADE não é subconta**
    Só organiza o dinheiro da empresa por finalidade e sócio responsável. Não existe agência, número nem conta separada, e o sócio não é dono de parte do saldo. Os saldos repetidos aqui somam os da CONTA (teste automatizado).

12. **substituto + janela**
    Entre início e fim da substituição, o substituto age como responsável (Maria cobre João nas férias), sem trocar o responsável. Fora da janela, só o responsável.

13. **ORCAMENTO**
    Teto de uso da responsabilidade por mês (competencia AAAA-MM). utilizado = comprometido + gasto no mês; devolve o valor se o pedido for cancelado. Sem orçamento cadastrado, não há teto.

14. **MOVIMENTO_RESERVA**
    Histórico de cada mexida na reserva: RESERVA (livre → responsabilidade), LIBERACAO (responsabilidade → livre), TRANSFERENCIA (entre responsabilidades). O saldo da responsabilidade é o resultado deles; status cobre pedidos aguardando aprovação.

15. **TARIFA versionada e imutável**
    Mudar o preço cria uma linha nova; o histórico não muda. Cada saída aponta para a linha usada, e no agendamento a tarifa é congelada: o valor comprometido é exatamente o que será debitado.

16. **STATUS + CONTA_STATUS_EVENTO**
    CONTA.status_id é o estado atual (leitura rápida); cada mudança grava um evento na mesma transação. O tempo em cada status é o intervalo até o evento seguinte.

17. **FK → ES e o nome autor_empresa_socio_id**
    A autoria aponta para o vínculo sócio-empresa, não para a pessoa: ela age como sócia daquela empresa, com as permissões dela. FK usa o nome da PK de destino.

18. **1 empresa : 1 conta**
    A UK em empresa_id limita a uma conta por empresa nesta entrega. Remover a UK é o caminho para várias contas (Open Finance).

Por que as entidades têm cores diferentes? A cor diz o papel da entidade no domínio

- **Principal** — empresa, conta e a movimentação financeira (TRANSACAO); o que o negócio é e faz.
- **Organização financeira** — como o dinheiro da empresa é dividido e planejado: responsabilidade, orçamento, pagamento programado.
- **Configuração / referência** — tabelas de consulta e regras: status, tarifa, permissão, regra de aprovação.
- **Associativa** — liga duas entidades (sócio ↔ empresa; aprovação sócio ↔ pedido). Aparece no diagrama de governança.
- **Histórico / eventos** — só se acrescenta; guarda o que aconteceu: status da conta, movimentos de reserva, auditoria.

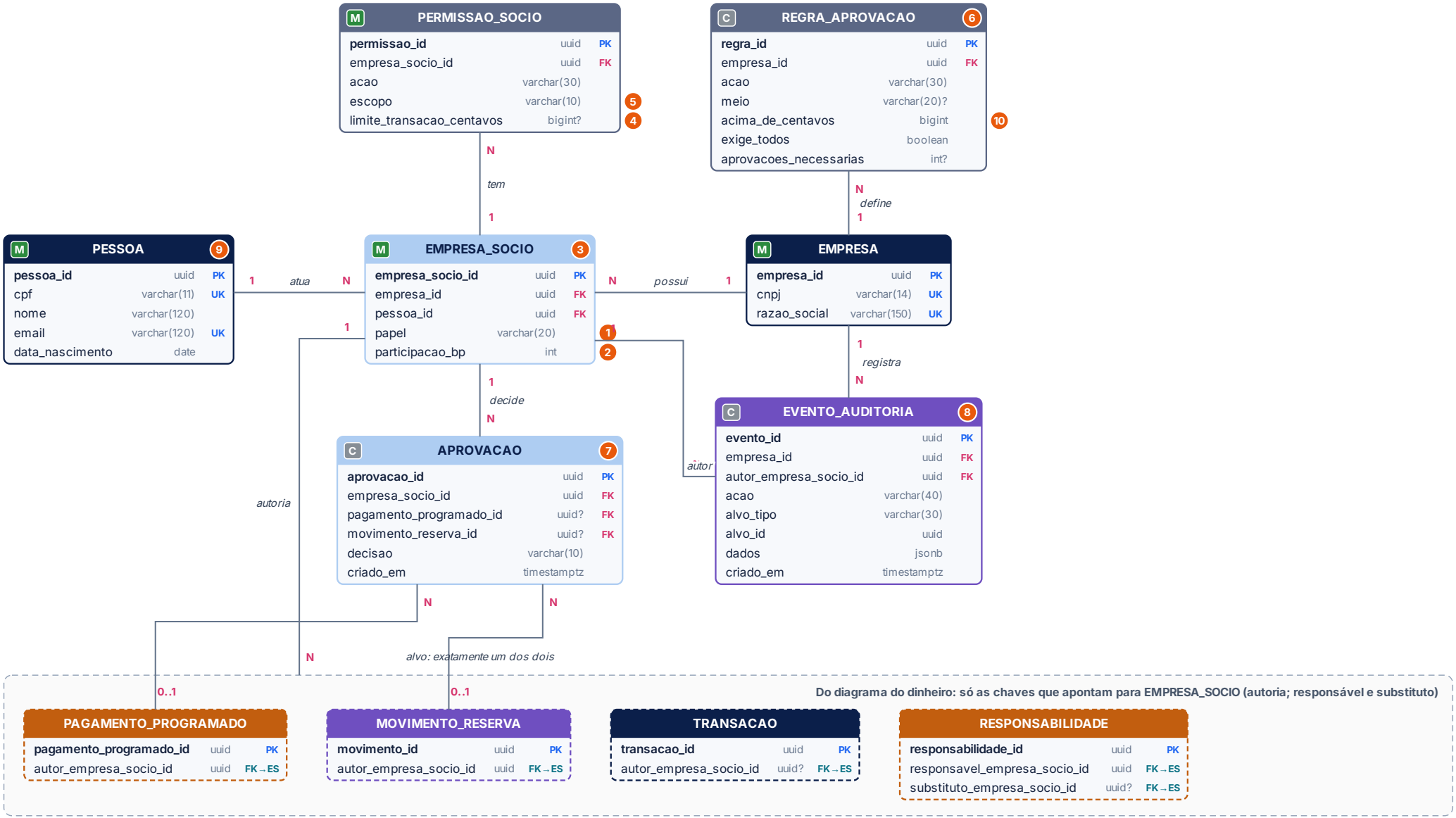
Selo no cabeçalho = prioridade na entrega (MoSCoW): M Must, S Should, C Could; cada selo é uma fatia que deixa a API funcionando (ver a RFC). As colunas de fatias futuras (responsabilidade_id, pagamento_programado_id, saldos reservado e comprometido) existem desde o início, anuláveis ou zeradas, e as FKs entram com a fatia. · PK chave primária · FK chave estrangeira · FK→ES chave estrangeira para EMPRESA_SOCIO (diagrama de governança) · UK chave única · ? aceita nulo · todos os valores em centavos.

Domínios: meio = PIX, TED, BOLETO, INTERNO · natureza = ENTRADA, SAIDA, INTERNA · MOVIMENTO_RESERVA.tipo = RESERVA, LIBERACAO, TRANSFERENCIA · MOVIMENTO_RESERVA.status = AGUARDANDO_APROVACAO, APLICADO, RECUSADO · PAGAMENTO_PROGRAMADO.statu = AGUARDANDO_APROVACAO, AGENDADO, EXECUTADO, RECUSADO, CANCELADO, FALHOU · STATUS.codigo = EM_APROVACAO, ATIVA, BLOQUEADA, ENCERRADA.

UK compostas: EMPRESA razao_social comparada sem maiúsculas nem espaços sobrando (uma razão social por CNPJ) · CONTA (agencia, numero) · RESPONSABILIDADE (conta_id, nome) · ORCAMENTO (responsabilidade_id, competencia) · TARIFA (meio, vigente_desde) · (autor_empresa_socio_id, chave_idempotencia) em TRANSACAO, PAGAMENTO_PROGRAMADO e MOVIMENTO_RESERVA · TRANSACAO (meio, referencia_externa) · PAGAMENTO_PROGRAMADO (meio, referencia_externa) parcial, só em AGUARDANDO_APROVACAO ou AGENDADO.

CHECK: valor_centavos > 0 · saldo_reservado ≥ 0, saldo_comprometido ≥ 0 e saldo_total ≥ saldo_reservado + saldo_comprometido · TRANSACAO coerente com natureza (ENTRADA sem origem; SAIDA sem destino; INTERNA com origem ≠ destino e sem contraparte; autor nulo só na ENTRADA) · MOVIMENTO_RESERVA coerente com tipo (RESERVA sem origem; LIBERACAO sem destino; TRANSFERENCIA com origem ≠ destino) · ORCAMENTO utilizado ≤ valor · substituto, início e fim juntos ou todos nulos, fim > início.

Índices de leitura: TRANSACAO (conta_origem_id, criado_em desc, transacao_id desc) e (conta_destino_id, criado_em desc, transacao_id desc), que sustentam o extrato paginado por cursor.


PERMISSAO_SOCIO
permissao_id uuid PK
empresa_socio_id uuid FK
acao varchar(30)
escopo varchar(10)
limite_transacao_centavos bigint?
5
4
REGRA_APROVACAO
6
regra_id uuid PK
empresa_id uuid FK
acao varchar(30)
meio varchar(20)?
acima_de_centavos bigint
exige_todos boolean
aprovacoes_necessarias int?
10
N
tem
1
N
define
1
PESSOA
9
pessoa_id uuid PK
cpf varchar(11) UK
nome varchar(120)
email varchar(120) UK
data_nascimento date
1
atua
N
1
EMPRESA_SOCIO
3
empresa_socio_id uuid PK
empresa_id uuid FK
pessoa_id uuid FK
papel varchar(20)
participacao_bp int
N
possui
1
EMPRESA
empresa_id uuid PK
cnpj varchar(14) UK
razao_social varchar(150) UK
1
2
1
registra
N
1
decide
N
APROVACAO
7
aprovacao_id uuid PK
empresa_socio_id uuid FK
pagamento_programado_id uuid? FK
movimento_reserva_id uuid? FK
decisao varchar(10)
criado_em timestamptz
autor
EVENTO_AUDITORIA
8
evento_id uuid PK
empresa_id uuid FK
autor_empresa_socio_id uuid FK
acao varchar(40)
alvo_tipo varchar(30)
alvo_id uuid
dados jsonb
criado_em timestamptz
autoria
N
N
N
alvo: exatamente um dos dois
0..1
0..1
Do diagrama do dinheiro: só as chaves que apontam para EMPRESA_SOCIO (autoria; responsável e substituto)
PAGAMENTO_PROGRAMADO
pagamento_programado_id uuid PK
autor_empresa_socio_id uuid FK→ES
MOVIMENTO_RESERVA
movimento_id uuid PK
autor_empresa_socio_id uuid FK→ES
TRANSACAO
transacao_id uuid PK
autor_empresa_socio_id uuid? FK→ES
RESPONSABILIDADE
responsabilidade_id uuid PK
responsavel_empresa_socio_id uuid FK→ES
substituto_empresa_socio_id uuid? FK→ES

Por quê? Legenda dos números do diagrama

1. **papel ≠ permissão**
   papel é a situação legal do sócio (SOCIO ou SOCIO_ADMINISTRADOR, do contrato social). Quem pode fazer o quê vem de PERMISSAO_SOCIO, não do papel: isso permite governança sob medida em vez de dois cargos fixos.
2. **participacao_bp**
   Participação societária em pontos-base (10000 = 100%; a soma por empresa é conferida no cadastro). Não é dono de saldo: o dinheiro é da empresa, o sócio só tem autorização para operar.
3. **Por que EMPRESA_SOCIO tem PK própria e não tem criado_em**
   A PK própria é o alvo de toda autoria (um FK simples, em vez de chave composta). A UK (empresa_id, pessoa_id) evita sócio duplicado. Data de entrada não é regra do domínio; mudanças relevantes ficam na auditoria.
4. **limite_transacao_centavos**
   Teto por operação daquele sócio naquela ação; nulo = sem teto. Vale antes de qualquer regra de aprovação: aprovação de outros sócios não libera um sócio acima do próprio limite.
5. **escopo**
   PROPRIAS: só nas responsabilidades em que é responsável (ou substituto na janela). TODAS: em qualquer responsabilidade e também no saldo livre da empresa.
6. **REGRA_APROVACAO**
   Diz a partir de que valor um pedido precisa da aprovação de outros sócios e de quantos (ou de todos: exige_todos, no lugar de usar nulo com sentido escondido). Vale a regra de maior acima_de que o valor ultrapassa; a regra com meio vence a sem meio.
7. **APROVACAO**
   Dois FKs opcionais, exatamente um preenchido, em vez de alvo_tipo + alvo_id: o banco garante que o alvo existe. A UK impede o mesmo sócio de votar duas vezes. O autor não aprova o próprio pedido, e uma recusa encerra o pedido.
8. **EVENTO_AUDITORIA**
   Só recebe linhas: registra quem mudou permissão, regra, substituição ou responsabilidade, com os dados da mudança. Sem FK para o alvo porque mistura vários tipos. Movimentações já têm autoria própria (TRANSACAO, pedidos e votos).
9. **PESSOA separada de EMPRESA_SOCIO**
   Dados da pessoa ficam num lugar só (CPF único); a mesma pessoa pode ser sócia de mais de uma empresa, com papel e permissões diferentes em cada uma.
10. **Como fica a regra do exemplo**
    "Qualquer sócio paga até R$ 1.000; acima disso, duas aprovações": permissão EXECUTAR_PAGAMENTO sem limite + uma regra (PAGAMENTO, acima_de_centavos = 100000, aprovacoes_necessarias = 2). R$ 1.000,00 exatos não exige aprovação.

Por que as entidades têm cores diferentes? A cor diz o papel da entidade no domínio

- **Principal** — empresa, pessoa e a movimentação financeira (TRANSACAO); o que o negócio é e faz.
- **Organização financeira** — como o dinheiro é dividido e planejado: responsabilidade, orçamento, pagamento programado.
- **Configuração / referência** — regras que a empresa define: permissão do sócio e regra de aprovação.
- **Associativa** — liga duas entidades: sócio↔empresa (EMPRESA_SOCIO) e sócio↔pedido (APROVACAO).
- **Histórico / eventos** — só se acrescenta; guarda o que aconteceu: movimentos de reserva, status da conta, auditoria.

Selo no cabeçalho = prioridade na entrega (MoSCoW): M Must, S Should, C Could. PERMISSAO_SOCIO é Must só com as permissões padrão geradas no cadastro; editar permissão e limite é Should. EMPRESA: cnpj e razao_social únicos (razão social comparada sem maiúsculas nem espaços sobrando). · PK chave primária · FK chave estrangeira · FK →ES chave estrangeira para EMPRESA_SOCIO · UK chave única · ? aceita nulo · bp = pontos-base (10000 = 100%). EMPRESA_SOCIO.papel = SOCIO ou SOCIO_ADMINISTRADOR.

PERMISSAO_SOCIO.acao = CONSULTAR, EXECUTAR_PAGAMENTO, APROVAR, GERIR_RESPONSABILIDADE, LIBERAR_RESERVA, GERIR_GOVERNANCA · escopo = PROPRIAS ou TODAS · UK (empresa_socio_id, acao). Ao cadastrar a empresa, o administrador recebe todas as ações (TODAS, sem limite) e o sócio comum recebe CONSULTAR; depois a empresa ajusta. Ao menos um sócio mantém GERIR_GOVERNANCA.

REGRA_APROVACAO.acao = PAGAMENTO, RESERVA, LIBERACAO_RESERVA · UK (empresa_id, acao, meio, acima_de_centavos) com NULLS NOT DISTINCT · CHECK exatamente um entre exige_todos verdadeiro e aprovacoes_necessarias preenchido.

APROVACAO.decisao = APROVADA, RECUSADA · UK (empresa_socio_id, pagamento_programado_id) e (empresa_socio_id, movimento_reserva_id) · CHECK exatamente um alvo preenchido. EMPRESA_SOCIO: CHECK participacao_bp entre 1 e 10000.

Fluxos

Regra para todos os fluxos. Toda operação que muda saldo ou pedido começa travando a `CONTA` (`SELECT … FOR UPDATE`; se houver duas contas, em ordem crescente de `conta_id`) e só então lê, confere e grava. Depois, ainda sob a trava da conta, trava a linha do pedido, quando existe. Assim nenhuma operação enxerga saldo ou estado velho, e dá um único `commit` antes do `return`.

Cadastro de empresa — caminho feliz

1. O resource valida o schema (`400`). O controller valida dígitos de CNPJ e CPF, idade ≥ 18, CPFs distintos, ao menos um sócio, ao menos um `SOCIO_ADMINISTRADOR` e `participacao_bp` somando 10000 (`422`). Só há sócios pessoas físicas.
2. Numa única transação de banco: cria cada `PESSOA` (ou reaproveita pelo CPF), insere `EMPRESA`, `CONTA` (saldos zerados, número gerado), um `EMPRESA_SOCIO` por sócio, as permissões padrão (administrador: todas as ações, `TODAS`, sem limite; sócio comum: `CONSULTAR`) e o evento inicial da conta (`EM_APROVACAO`).
3. Um único `commit`; resposta `201` com `empresa_id` e `conta_id`. A conta só movimenta dinheiro depois que o back-office a aprova.

Cadastro de empresa — falha: o mesmo CNPJ (ou a mesma razão social) em duas requisições simultâneas

1. As duas passam na checagem "o CNPJ ou a razão social já existem?", porque nenhuma gravou ainda. A razão social é única porque cada CNPJ tem uma só, e a comparação ignora maiúsculas e espaços sobrando.
2. A segunda viola `UNIQUE(cnpj)` ou `UNIQUE(razao_social)` no `INSERT`; o controller faz rollback do cadastro inteiro (nem pessoas novas nem conta ficam gravadas) e responde `409`.
3. A checagem prévia só poupa trabalho no caso comum; a garantia é a restrição do banco.

Transação imediata (saída ou transferência interna) — caminho feliz

1. O middleware confere o `INTERNAL-TOKEN`; o resource valida o corpo e a `contraparte` pelo JSON Schema do meio (`400`). O controller confere `X-Pessoa-Id` (`401`), a conta de origem (`404`) e a permissão `EXECUTAR_PAGAMENTO` do sócio, incluindo o escopo sobre a responsabilidade informada (`403`).
2. Trava as contas do banco envolvidas, em ordem crescente de `conta_id` (evita deadlock entre A→B e B→A).
3. Com a trava, busca `(autor, chave)`: mesmo hash devolve a transação gravada (`200`); hash diferente é `409`. No boleto busca também `(meio, referencia_externa)`; se já existe, `409`.
4. Se o destino é externo, busca a `TARIFA` vigente do meio (maior `vigente_desde` menor ou igual a agora); transferência interna não paga tarifa. `total = valor + tarifa`.
5. Confere nesta ordem (`422`): contas de origem e de destino interno `ATIVA`; `valor_centavos` dentro do `limite_transacao_centavos` do sócio; nenhuma regra de aprovação alcança o valor (senão `APROVACAO_NECESSARIA`); orçamento do mês da responsabilidade, se houver; e dinheiro disponível: o `reservado` da responsabilidade, se informada, ou o saldo livre da conta.
6. `INSERT` em `TRANSACAO` com `natureza`, `saldo_origem_apos`, `saldo_destino_apos`, `tarifa_id`, autor, responsabilidade, categoria e descrição; baixa o `saldo_total` (e o `reservado` da responsabilidade, se for o caso) e credita a conta de destino interna; soma o `utilizado` do orçamento.
7. Um único `commit`; resposta `201` com `transacao_id`, valor, tarifa e saldo após.

Transação — falha: dois pagamentos simultâneos sobre o mesmo saldo

1. Saldo livre de R$ 100,00; chegam dois Pix de R$ 80,00 com chaves diferentes. As duas disputam a trava da conta (passo 2).
2. A requisição A trava primeiro, grava e dá `commit`: livre R$ 20,00. B espera.
3. B obtém a trava, relê o saldo já atualizado (R$ 20,00 < R$ 80,00), faz rollback e recebe `422 SALDO_LIVRE_INSUFICIENTE`. Nada de B fica gravado.

4. O mesmo vale para dinheiro comprometido: um pagamento programado já tirou o valor do livre, então um Pix imediato não gasta o mesmo dinheiro. Reenviar com a mesma chave é uma nova tentativa legítima, porque a primeira não deixou registro.

Transação — falha: resposta perdida e retentativa

1. A requisição A deu `commit`, mas o cliente não recebeu a resposta (timeout) e reenvia a mesma chave e o mesmo corpo.
2. O passo 3 encontra `(autor, chave)` com o mesmo hash e devolve `200` com o mesmo `transacao_id`; o saldo não muda. Se o corpo vier alterado: `409`.
3. Se as duas chegarem ao mesmo tempo, serializam pela trava do passo 2; `UNIQUE(autor_empresa_socio_id, chave_idempotencia)` é a última defesa.

Transação — falha: valor acima do limite do sócio, ou que exige aprovação

1. Um sócio com limite de R$ 5.000,00 pede um Pix de R$ 8.000,00, com saldo suficiente. O passo 5 recusa com `422 LIMITE_SOCIO_EXCEDIDO` e faz rollback. Um sócio com limite maior, ou sem limite, consegue; cada pagamento fica identificado pelo autor no extrato.
2. Se a empresa tem a regra "acima de R$ 1.000,00, duas aprovações" e o Pix é de R$ 1.500,00, o passo 5 recusa com `422 APROVACAO_NECESSARIA`: um pagamento que precisa de aprovação não é executado na hora, e o cliente deve usar `/pagamento_programado`.

Transação — falha: o mesmo boleto pago duas vezes

1. Duas requisições chegam com o mesmo `codigo_barras` (`referencia_externa`), chaves diferentes e de empresas diferentes: travam contas diferentes, e nenhuma enxerga a outra. Na mesma empresa, a trava do passo 2 faz a segunda ver a primeira no passo 3.
2. A primeira insere e dá `commit`. A segunda viola `UNIQUE(meio, referencia_externa)`, faz rollback do pagamento inteiro e recebe `409` (boleto já pago); o saldo dela fica intacto.

Entrada de fora do banco — caminho feliz e falha

1. O gateway simulado chama `POST /transacao` sem `conta_origem_id`, com `conta_destino_id`, `contraparte` e `referencia_externa` (identificador do parceiro), e sem `X-Pessoa-Id`.
2. O controller trava a conta de destino e busca `(meio, referencia_externa)`: mesmo hash devolve `200`, hash diferente é `409`. Se a conta não está `ATIVA`, `422 CONTA_NAO_ATIVA`.
3. Credita o `saldo_total` (cai no livre), insere a `TRANSACAO` `ENTRADA` com `saldo_destino_apos` (sem tarifa e sem autor) e dá `commit`; resposta `201`.
4. Falha: a mesma notificação do parceiro chega duas vezes ao mesmo tempo. Serializam pela trava, a segunda vê a primeira e recebe `200`, e o crédito acontece uma só vez.

Reserva de dinheiro — caminho feliz e falha

1. Maria pede `POST /movimento_reserva` do tipo `RESERVA` de R$ 500,00 para Freelas. O controller confere `GERIR_RESPONSABILIDADE`, o limite do sócio e a regra de aprovação (`RESERVA`).
2. Sem regra que alcance o valor: trava a conta, confere idempotência e `CONTA_NAO_ATIVA`, e confere que o **livre** (`total − reservado − comprometido`) cobre os R$ 500,00. Soma 500 ao `reservado` da responsabilidade e da conta, grava o movimento `APLICADO` com `decidido_em` e dá `commit`. O `saldo_total` não muda: o dinheiro continua na conta.
3. Com regra: o movimento nasce `AGUARDANDO_APROVACAO` com `aprovacoes_exigidas`, sem mexer em saldo; o efeito só acontece na última aprovação, que repete a conferência sob a trava.
4. Falha: o livre é de R$ 300,00, porque pagamentos programados já comprometeram o resto. A requisição recebe `422 SALDO_LIVRE_INSUFICIENTE` e nada é gravado. Na liberação, o limite é o `reservado` da responsabilidade (`RESERVA_INSUFICIENTE`); dinheiro comprometido não pode ser liberado.

Pagamento programado e aprovação — caminho feliz

1. João programa um Pix de R$ 400,00 para sexta, usando a responsabilidade Freelas. O resource valida o corpo e a `contraparte` (`400`); o controller confere permissão e escopo (`403`), a data (`DATA_NO_PASSADO`) e o limite do sócio.
2. Trava a conta e confere idempotência. Busca a `TARIFA` vigente e a congela no pedido, para que o valor comprometido seja exatamente o que será debitado. Confere orçamento do mês e que o `reservado` da responsabilidade cobre `valor + tarifa` (ou o livre, sem responsabilidade).
3. Resolve a regra de aprovação (a de maior `acima_de` que o valor ultrapassa; a regra com `meio` vence a sem `meio`) e grava `aprovacoes_exigidas` no pedido: zero sem regra; N pela quantidade; ou o número de sócios elegíveis, se for "todos". Elegíveis são os sócios com `APROVAR` no escopo da responsabilidade, exceto o autor; se forem menos que o exigido, `422 REGRA_INATINGIVEL`.
4. Move `valor + tarifa` de `reservado` para `comprometido` (na responsabilidade e na conta) e soma o `utilizado` do orçamento. Grava o pedido `AGENDADO`, ou `AGUARDANDO_APROVACAO` se `aprovacoes_exigidas > 0`. Um único `commit`; `201`.
5. Cada aprovador chama `/aprovacao`. O controller trava a conta, depois o pedido, e relê o status (`PEDIDO_NAO_PENDENTE` se já mudou). Confere elegibilidade e que não é o autor, insere a aprovação e conta os votos; ao atingir o exigido, o pedido vira `AGENDADO`. Nenhum saldo muda na aprovação, porque o dinheiro já estava comprometido.

Pagamento programado — falha: recusa, cancelamento ou duas aprovações simultâneas

1. Uma recusa, ou o cancelamento pelo autor, trava a conta e o pedido, devolve `valor + tarifa` de `comprometido` para `reservado` (ou para o livre, sem responsabilidade), subtrai do `utilizado` do orçamento e marca `RECUSADO` ou `CANCELADO`, tudo no mesmo `commit`.
2. Duas aprovações ao mesmo tempo disputam a trava da conta: a segunda espera, vê os votos da primeira e o novo status. Se a primeira completou o número exigido, a segunda recebe `422 PEDIDO_NAO_PENDENTE`. O mesmo sócio votando duas vezes é barrado por `UNIQUE(empresa_socio_id, pagamento_programado_id)`.
3. Se uma permissão mudar depois e faltarem aprovadores elegíveis, o pedido não anda sozinho: o autor cancela e recomeça.

Execução do pagamento programado — caminho feliz e falha

1. Na sexta o agendador chama `POST /pagamento_programado/{id}/execucao`. O controller lê o pedido para achar a conta, trava a conta, trava o pedido e relê o status.
2. Se está `AGENDADO` e a data chegou, insere a `TRANSACAO` `SAIDA` com `pagamento_programado_id`, o mesmo autor, a tarifa congelada e `saldo_origem_apos`; baixa `comprometido` e `saldo_total`; marca o pedido `EXECUTADO`; `commit`; `201`.
3. Falha, execução repetida: o pedido já está `EXECUTADO` e `UNIQUE(pagamento_programado_id)` impede a segunda transação; a rota devolve `200` com a mesma `transacao_id`.
4. Falha, conta bloqueada: `422 CONTA_NAO_ATIVA`, o pedido continua `AGENDADO` e o dinheiro continua comprometido, para o agendador tentar de novo depois. Boleto vencido: o pedido vira `FALHOU` e devolve o dinheiro.
5. Cancelamento e execução ao mesmo tempo serializam pela trava: quem chega primeiro vence, e o outro vê o estado final (`422 NAO_AGENDADO` ou `422 JA_EXECUTADO`).

Extrato — como o sócio vê o dinheiro

1. O controller confere que a pessoa é sócia da empresa dona da conta (`401`, `403`) e lê os filtros. Filtro malformado ou `limite` fora de 1–100 é `400`.
2. Seleciona as transações em que a conta é origem ou destino, ordenadas por `(criado_em, transacao_id)` decrescente, com cursor e não `OFFSET`: inserções durante a leitura fariam linhas repetirem ou sumirem. Os índices por conta e `criado_em` sustentam a consulta.

3. Cada item traz data, natureza, meio, contraparte, valor com sinal para aquela conta, tarifa, categoria, descrição, nome do sócio autor, responsabilidade, o pedido que originou a transação (para chegar às aprovações) e o saldo após (`saldo_origem_apos` se a conta é a origem, `saldo_destino_apos` se é o destino).
4. O saldo é reconstruível: entradas menos saídas e tarifas dão `saldo_total_centavos`, e isso é um teste automatizado, assim como `reservado` da conta igual à soma das responsabilidades.

Mudança de status da conta durante um pagamento

1. O back-office chama `PATCH /conta/{id}/status` com `BLOQUEADA`; o controller trava a conta com a mesma trava dos pagamentos, valida a transição, atualiza `status_id` e insere o evento na mesma transação. Para `ENCERRADA`, exige saldo total zero, o que implica nada reservado nem comprometido.
2. Se um pagamento já tinha a trava, o bloqueio espera: aquele pagamento é válido e os seguintes são recusados. Se o bloqueio veio primeiro, o pagamento lê `BLOQUEADA` e recebe `422 CONTA_NAO_ATIVA`.
3. Repetir o `PATCH` com o mesmo status devolve `200` sem novo evento, então o histórico não acumula duplicatas.

Principal desafio

- **Qual é:** garantir que requisições concorrentes e retentativas sobre a mesma conta nunca gastem o mesmo dinheiro duas vezes, nem dupliquem um débito, um crédito ou uma execução, e que o dinheiro em estados diferentes (livre, reservado, comprometido) e as aprovações no meio do caminho mantenham saldo e extrato coerentes.
- **Por que é difícil:** o padrão "consulta se pode e depois escreve" (o do controller de exemplo do base-service) tem condição de corrida: com `READ COMMITTED`, duas requisições leem R$ 100 e ambas debitam R$ 80. Uma retentativa após timeout parece uma operação nova. Uma transferência interna escreve em duas contas, que não podem ficar pela metade nem travar uma à outra em ordens opostas. E agora um pedido atravessa várias etapas (comprometer, aprovar, executar ou cancelar), cada uma podendo chegar ao mesmo tempo que outra sobre o mesmo dinheiro.
- **Como o desenho resolve:** (1) uma regra única: toda operação trava a `CONTA` com `SELECT … FOR UPDATE`, em ordem de `conta_id`, antes de ler saldo, estado ou idempotência, e depois trava a linha do pedido, o que serializa tudo o que toca o dinheiro da conta e elimina deadlock; (2) comprometer o dinheiro no agendamento, na mesma transação, garante que a execução futura não falha por saldo e que o mesmo dinheiro não é prometido duas vezes; (3) `UNIQUE(autor_empresa_socio_id, chave_idempotencia)`, `UNIQUE(meio, referencia_externa)`, `UNIQUE(pagamento_programado_id)` e `UNIQUE(empresa_socio_id, alvo)` são a defesa final contra duplicata, e o `hash_requisicao` separa repetição legítima (`200`) de abuso da chave (`409`); (4) tudo fica numa transação de banco, com um único `commit` no controller; (5) os `CHECK` de saldo (`total ≥ reservado + comprometido ≥ 0`) são a última barreira; (6) o cenário "dois pagamentos simultâneos, um só passa", a execução repetida e as invariantes "saldo = entradas − saídas − tarifas" e "reservado da conta = soma das responsabilidades" são testes automatizados em `pytest`.